

Learn Low-Level Design in 25 Days: Day 9—Adapter, Decorator, and Strategy Patterns Explained

(Part of the Learn LLD in 25 Days Series—save the list here)

TechPreneur

Published October 21, 2025

Updated October 22, 2025

Free: No

Read on Freedium · original

Contents

When Interfaces Don't Match and Classes Need to Cooperate 0

1. Adapter Pattern — Making Things Fit 0

2. Decorator Pattern — Add Features Without Breaking Things 0

3. Strategy Pattern — Switch Behavior at Runtime 0

Try It Yourself 0

Summary — The Patterns in a Nutshell 0

When Interfaces Don't Match and Classes Need to Cooperate

You've just joined a new project. You open the codebase and realize half of it uses a new payment API while the rest still depends on an older one. Both work fine — but their interfaces don't match. You can't just "plug" one into the other.

Or maybe you're building a notification system. You want to send messages by email, SMS, and push — but each channel needs slightly different behavior. Duplicating logic feels wrong, but subclassing everything is even worse.

Welcome to **Day 9** of Learn Low-Level Design in 25 Days. Today we're diving into three patterns that make systems flexible, extensible, and maintainable: **Adapter**, **Decorator**, and **Strategy**.



By the end of this article, you'll know how to:

- Make incompatible interfaces work together (Adapter)
- Add behavior dynamically without touching existing code (Decorator)
- Switch algorithms or behaviors at runtime (Strategy)

Let's get practical.

1. Adapter Pattern — Making Things Fit

Problem: You want to reuse existing code, but its interface doesn't match what you need.

Analogy: Think of a **travel plug adapter** — your laptop charger works in the US, but not in Europe. The adapter lets you use the same charger without modifying it.

In code:

Imagine your app sends notifications, but the new SMS service provider has a completely different method signature.

```
// Existing interface
interface Notifier {
    void send(String message);
}

// New external service
class TwilioService {
    void pushSMS(String text, String phoneNumber) {
        System.out.println("Sending SMS via Twilio: " + text);
    }
}

// Adapter to make Twilio fit our Notifier interface
class SMSAdapter implements Notifier {
    private TwilioService twilio = new TwilioService();

    @Override
    public void send(String message) {
        twilio.pushSMS(message, "+1234567890");
    }
}
```

Now your existing `NotificationService` can keep calling `.send()` — and it'll just work.

Class Diagram:

```
Client ---> Target (Notifier)
              ^
              |
          Adapter (SMSAdapter) ---> Adaptee (TwilioService)
```

When to Use Adapter:

- You're integrating third-party APIs or legacy code.

- You want to avoid rewriting existing classes.
- You need to make two incompatible interfaces cooperate.

2. Decorator Pattern — Add Features Without Breaking Things

Problem: You want to add behavior to an object dynamically — without changing its class or creating tons of subclasses.

Analogy: Think of **a coffee order**: You start with plain coffee, then add milk, sugar, or whipped cream. Each "decoration" enhances the base object without altering it.

In code:

Let's take a basic `Notifier`, and we'll "decorate" it to send multiple types of notifications.

```
interface Notifier {
    void send(String message);
}

class BasicNotifier implements Notifier {
    public void send(String message) {
        System.out.println("Sending basic notification: " + message);
    }
}

class EmailDecorator implements Notifier {
    private Notifier notifier;

    EmailDecorator(Notifier notifier) {
        this.notifier = notifier;
    }

    public void send(String message) {
        notifier.send(message);
        System.out.println("Sending email: " + message);
    }
}

class SlackDecorator implements Notifier {
    private Notifier notifier;

    SlackDecorator(Notifier notifier) {
        this.notifier = notifier;
    }

    public void send(String message) {
        notifier.send(message);
        System.out.println("Sending Slack message: " + message);
    }
}

// Usage
Notifier notifier = new SlackDecorator(new EmailDecorator(new BasicNotifier()));
notifier.send("Server down!");
```

Output:

```
Sending basic notification: Server down!
Sending email: Server down!
Sending Slack message: Server down!
```

Class Diagram:

```
Component (Notifier)
  |
  +-- ConcreteComponent (BasicNotifier)
  |
  +-- Decorator (EmailDecorator, SlackDecorator)
```

When to Use Decorator:

- You want to add features dynamically (logging, monitoring, etc.).
- You don't want to modify existing classes.
- You want more flexibility than inheritance offers.

3. Strategy Pattern — Switch Behavior at Runtime

Problem: You have multiple ways to perform a task, and you want to choose the right one dynamically.

Analogy: Think of **navigation apps** — you can choose the fastest route, the shortest route, or avoid tolls. The algorithm changes, but the interface remains the same.

In code:

```
interface PaymentStrategy {
    void pay(int amount);
}

class CreditCardPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Paid " + amount + " using Credit Card");
    }
}

class PayPalPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Paid " + amount + " using PayPal");
    }
}

class PaymentContext {
    private PaymentStrategy strategy;

    public PaymentContext(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    public void setStrategy(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    public void checkout(int amount) {
        strategy.pay(amount);
    }
}

// Usage
PaymentContext context = new PaymentContext(new CreditCardPayment());
context.checkout(500);

context.setStrategy(new PayPalPayment());
context.checkout(500);
```

Output:

```
Paid 500 using Credit Card
Paid 500 using PayPal
```

Class Diagram:

```
Context (PaymentContext)
    |
    +--> Strategy (PaymentStrategy)
              |
              +--> ConcreteStrategyA (CreditCardPayment)
              +--> ConcreteStrategyB (PayPalPayment)
```

When to Use Strategy:

- You have multiple algorithms or behaviors for the same operation.
- You want to switch logic at runtime.
- You want to avoid giant `if-else` or `switch` statements.

Try It Yourself

1. **Adapter Challenge:** You're integrating a new email API that uses `.deliverMail()` instead of `.sendEmail()`. Sketch an adapter class that lets your `NotificationService` continue using `.send()`.
2. **Decorator Challenge:** Add a `PushNotificationDecorator` to the notifier system above.
3. **Strategy Challenge:** Implement a `DiscountStrategy` for an e-commerce cart — switch between `FestivalDiscount`, `MemberDiscount`, and `NoDiscount`.

Summary — The Patterns in a Nutshell

Adapter: Makes incompatible interfaces work together. **Decorator:** Adds new behavior dynamically without changing code. **Strategy:** Lets you switch algorithms or behaviors at runtime.

Key Takeaways:

- Composition often beats inheritance in flexibility.
- Patterns help you **extend**, not **rewrite**.
- Clean, modular designs reduce ripple effects when requirements change.

When to Avoid:

- If a simpler abstraction (like direct composition or a config flag) solves the problem.
- If you're adding patterns just for the sake of "using patterns."

Up Next — Day 10: Tomorrow, we'll dive into **Observer, Publisher-Subscriber, and Event-Driven Design** — the foundation behind systems like Slack, Trello, and Gmail.

Follow the full Learn Low-Level Design in 25 Days series here: https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8 (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)